

DP Transition Diagram

Absolutely. The key thing to understand is that this is **0/1 Knapsack with two resources**:

- z = number of zeros available
- o = number of ones available
- each string = one item
- each item can be selected **at most once**
- selecting a string gives +1 to the answer

Let's visualize the whole process.

1. What does $dp[z][o]$ mean?

Your code uses:

```
C++
dp[z][o]
```

Meaning:

Using at most z zeros and o ones, what is the maximum number of strings I can select?

For example:

```
dp[3][4] = 2
```

means:

```
Available:
0s = 3
1s = 4
Maximum strings we can form = 2
```

2. Initial DP table

Suppose:

```
strs = ["10", "0001", "111001", "1", "0"]
m = 5
n = 3
```

Initially:

```
dp[z][o] = 0
```

So the table is:

		ones →			
		0	1	2	3
z = 0	+	0	0	0	0
z = 1		0	0	0	0
z = 2		0	0	0	0
z = 3		0	0	0	0
z = 4		0	0	0	0
z = 5		0	0	0	0

There are currently **no strings processed**, so answer is 0 everywhere.

3. Process the first string

Current string:

```
"10"
```

Count:

```
zeros = 1
ones = 1
```

So this string costs:

```

      "10"
     /  \
    /    \
   1 zero 1 one

```

If we take it:

```

answer += 1
zero capacity -= 1
one capacity -= 1

```

4. The transition

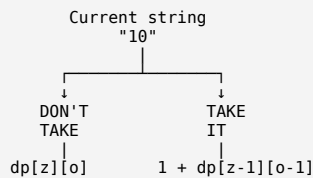
Your most important line is:

```

C++
dp[z][o] = max(
    dp[z][o],
    1 + dp[z - zeros][o - ones]
);

```

This represents **two choices**.



Then:

```

dp[z][o]
=
max(
    DON'T TAKE,
    TAKE
)

```

5. Why dp[z-1][o-1]?

Suppose we're calculating:

```
dp[3][2]
```

We have:

```

3 zeros
2 ones

```

and current string "10" requires:

```

1 zero
1 one

```

If we take "10":

```

Available before:
zeros = 3
ones  = 2
      ↓ TAKE "10"
remaining:
zeros = 3 - 1 = 2
ones  = 2 - 1 = 1

```

Therefore:

```
dp[3][2]
```

becomes:

```
1 + dp[2][1]
```

Diagram:

```
dp[3][2]
```

```

      |
Take "10"
      |
+1 string
      |
    dp[2][1]
  
```

6. First string complete

Because "10" requires (1 zero, 1 one), after processing it:

	ones →			
	0	1	2	3
z = 0	0	0	0	0
z = 1	0	1	1	1
z = 2	0	1	1	1
z = 3	0	1	1	1
z = 4	0	1	1	1
z = 5	0	1	1	1

Why?

For example:

```

dp[3][2]
DON'T TAKE = 0
TAKE:
1 + dp[2][1]
= 1 + 0
= 1
dp[3][2] = 1
  
```

7. Now process "0001"

Next:

```
s = "0001"
```

Count:

```

zeros = 3
ones = 1
  
```

So:

```

"0001"
  ↓
3 zeros
1 one
  
```

Now the transition becomes:

```

C++
dp[z][o] =
    max(
        dp[z][o],
        1 + dp[z-3][o-1]
    );
  
```

8. Important transition example

Let's calculate:

```
dp[5][2]
```

Current string:

```
"0001"
```

requires:

```

3 zeros
1 one
  
```

Choice 1 — Don't take

```
dp[5][2]
```

Current value from previous strings:

```
1
```

Choice 2 — Take

Remaining capacity:

```
zeros = 5 - 3 = 2
ones  = 2 - 1 = 1
```

Therefore:

```
1 + dp[2][1]
```

We already know:

```
dp[2][1] = 1
```

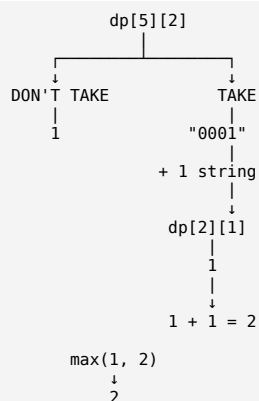
So:

```
Take = 1 + 1
      = 2
```

Therefore:

```
dp[5][2] = max(1, 2)
          = 2
```

Visually:



9. Why are the loops going backwards?

This part is **extremely important**:

```
C++
for (int z = m; z >= zeros; z--) {
    for (int o = n; o >= ones; o--) {
```

Why?

Because each string can be used **only once**.

Suppose we went forward:

```
z = zeros → m
o = ones  → n
```

Then after updating:

```
dp[z][o]
```

we might later use that newly updated state again for the **same string**.

That would accidentally allow:

```
"10"
"10"
"10"
...
```

multiple times.

Backward traversal prevents that.

10. Visual meaning of backward traversal

Suppose current string needs:

```
zeros = 1
ones = 1
```

We're calculating:

```
dp[3][3]
```

Transition:

```
dp[3][3]
  ↓
1 + dp[2][2]
```

The source:

```
dp[2][2]
```

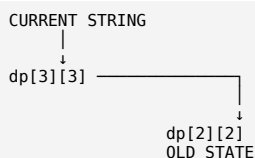
is **smaller** than the current state.

Because we're moving:

```
3 → 2 → 1
```

we are always reading a state that existed **before the current string was processed**.

Conceptually:



We never let the current string create:

```
dp[2][2]
  ↓
dp[3][3]
  ↓
dp[4][4]
```

during the same iteration.

11. Complete example

Let's use a smaller example to see the table evolve clearly:

```
strs = ["10", "0", "1"]
m = 2
n = 1
```

Resource requirements:

String	zeros	ones
"10"	1	1
"0"	1	0
"1"	0	1

Initial:

		ones	
		0	1
zero	0	0	0
	1	0	0
	2	0	0
	3	0	0

12. Process "10"

Cost:

(1 zero, 1 one)

Transition:

```
dp[z][o]
=
max(
    dp[z][o],
    1 + dp[z-1][o-1]
)
```

Table:

		ones	
		0	1
zero	0	0	0
	1	0	1
	2	0	1

Meaning:

```
dp[1][1] = 1
dp[2][1] = 1
```

We can take "10" once.

13. Process "0"

Cost:

(1 zero, 0 ones)

Transition:

```
dp[z][o]
=
max(
    dp[z][o],
    1 + dp[z-1][o]
)
```

Let's calculate:

```
dp[2][1]
```

Don't take:

```
dp[2][1] = 1
```

Take:

```
1 + dp[1][1]
```

and:

```
dp[1][1] = 1
```

so:

```
1 + 1 = 2
```

Therefore:

```
dp[2][1] = 2
```

Table:

		ones	
		0	1
zero	0	0	0
	1	1	1
	2	1	2

The 2 means:

```
"10" + "0"

zeros used = 1 + 1 = 2
ones used  = 1 + 0 = 1

total strings = 2
```

14. Process "1"

Cost:

```
(0 zeros, 1 one)
```

Transition:

```
dp[z][o]
=
max(
    dp[z][o],
    1 + dp[z][o-1]
)
```

Calculate:

```
dp[2][1]
```

Don't take:

```
dp[2][1] = 2
```

Take:

```
1 + dp[2][0]
```

We have:

```
dp[2][0] = 1
```

Therefore:

```
Take = 1 + 1 = 2
```

So:

```
dp[2][1] = max(2, 2)
           = 2
```

Final table:

		ones	
		0	1
zero	0	0	1
	1	1	2
	2	1	2
	2	1	2

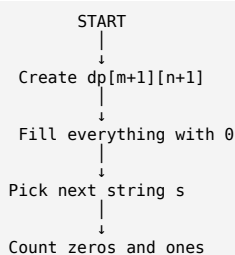
Final answer:

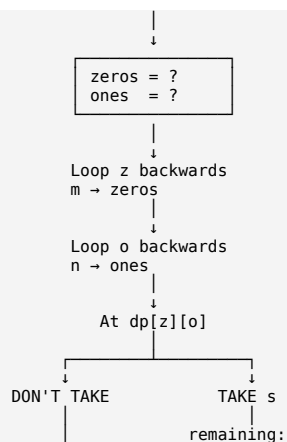
```
C++
return dp[m][n];
```

Therefore:

```
dp[2][1] = 2
```

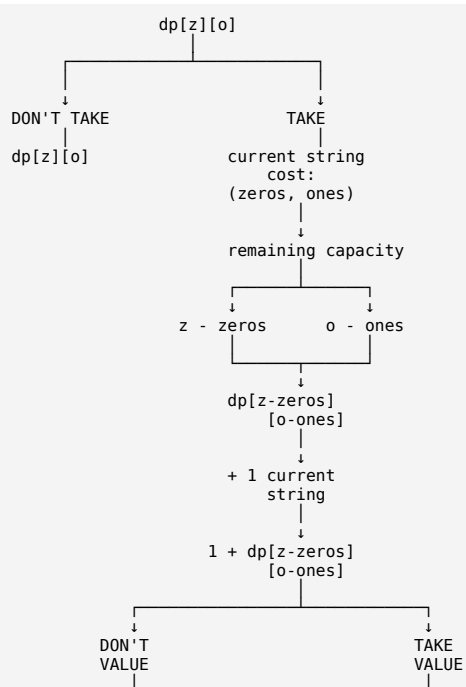
15. The entire algorithm as a flow diagram





16. The most important picture: DP transition

Every cell is basically doing this:



17. Think of the 2D table as a resource map

This is the easiest mental model.

		AVAILABLE ONES			
		0	1	2	3
ZEROS	0				
	1				
	2				
	3				
	4				

Each cell answers:

"With THIS many zeros and THIS many ones,
how many strings can I select?"

When a string costs (2 zeros, 1 one):

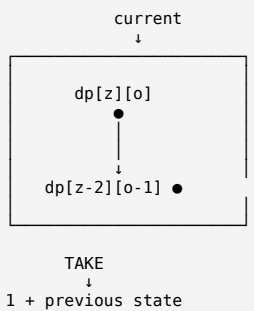
Current cell:

dp[z][o]
↓ move back by (2,1)
dp[z-2][o-1]

Then:

```
dp[z][o] = max(
    old value,
    1 + previous state
)
```

So you can literally visualize every transition as an arrow:



18. Connection to normal 0/1 Knapsack

Normal knapsack has:

```
dp[capacity]
```

and transition:

```
C++
dp[c] = max(
    dp[c],
    value + dp[c-weight]
);
```

Here you have **two capacities**:

```
zero capacity
one capacity
```

So it becomes:

Normal:

capacity
↓
dp[c]

This problem:

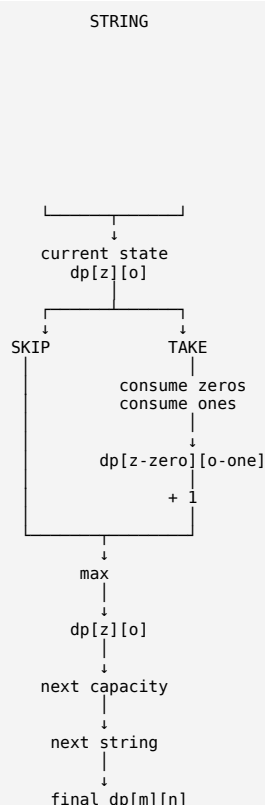
zeros ones
↓ ↓
dp[z][o]

And the transition becomes:

```
C++
dp[z][o] =
    max(
        dp[z][o], // don't take
        1 + dp[z-zeros][o-ones] // take
    );
```

That's the entire idea.

Final mental model



The backward loops + `dp[z-zeros][o-ones]` are what make this a 0/1 knapsack, ensuring every string is counted at most once.

Yes. Let's do the **complete dry run** of your exact example, including **every DP table update**.

Small correction: you wrote "0" in the last string. I'm assuming you mean "0" (zero).

If it is actually uppercase "0", your code treats it as a 1 because of the `else`.

```
C++
strs = ["10", "0001", "1", "0"]

m = 3 // maximum zeros
n = 2 // maximum ones
```

1. First understand the input

Each string has a cost:

String	Zeros	Ones	Value
"10"	1	1	1
"0001"	3	1	1
"1"	0	1	1
"0"	1	0	1

We have:

```
Maximum zeros = 3
Maximum ones = 2
```

So our DP table is:

```
dp[zeros][ones]
```

2. Initial DP table

Initially everything is 0.

		ONES →		
		0	1	2
ZERO	0	0	0	0
	1	0	0	0
	2	0	0	0
	3	0	0	0

Meaning:

```
dp[2][1] = 0
```

means:

With 2 zeros and 1 one available, we currently can select 0 strings.

3. Process string "10"

First:

```
C++
s = "10"
```

Count:

```
zeros = 1
ones = 1
```

So the transition is:

```
C++
dp[z][o] = max(
    dp[z][o],
    1 + dp[z - 1][o - 1]
);
```

And we loop backwards:

```
C++
z = 3 → 1
o = 2 → 1
```

Calculate dp[3][2]

Current state:

```
dp[3][2]
```

Don't take "10"

```
dp[3][2] = 0
```

Take "10"

It consumes:

```
1 zero
1 one
```

Remaining:

```
3 - 1 = 2 zeros
2 - 1 = 1 one
```

So:

```
1 + dp[2][1]
= 1 + 0
```

```
= 1
```

Therefore:

```
dp[3][2] = max(0, 1)
          = 1
```

Calculate dp[3][1]

```
Don't take = 0
```

```
Take:
1 + dp[2][0]
= 1 + 0
= 1
```

Therefore:

```
dp[3][1] = 1
```

Calculate dp[2][2]

```
Don't take = 0
```

```
Take:
1 + dp[1][1]
= 1 + 0
= 1
```

Therefore:

```
dp[2][2] = 1
```

Calculate dp[2][1]

```
Don't take = 0
```

```
Take:
1 + dp[1][0]
= 1
```

Therefore:

```
dp[2][1] = 1
```

Calculate dp[1][2]

```
Don't take = 0
```

```
Take:
1 + dp[0][1]
= 1
```

Therefore:

```
dp[1][2] = 1
```

Calculate dp[1][1]

```
Don't take = 0
```

```
Take:
1 + dp[0][0]
= 1
```

Therefore:

```
dp[1][1] = 1
```

4. DP table after "10"

```
      ONES →
0     1     2
```

ZERO	0	0	0	0
	1	0	1	1
	2	0	1	1
	3	0	1	1

Important:

```
dp[3][2] = 1
```

because we can take "10".

5. Process "0001"

Now:

```
s = "0001"
zeros = 3
ones = 1
```

Transition becomes:

```
C++
dp[z][o] = max(
    dp[z][o],
    1 + dp[z - 3][o - 1]
);
```

Because we need at least:

```
3 zeros
1 one
```

Since $m = 3$, the only possible zero capacity is:

```
z = 3
```

And:

```
o = 2 → 1
```

Calculate dp[3][2]

Don't take "0001"

Current:

```
dp[3][2] = 1
```

Take "0001"

It consumes:

```
3 zeros
1 one
```

Remaining:

```
3 - 3 = 0 zeros
2 - 1 = 1 one
```

Therefore:

```
1 + dp[0][1]
```

Currently:

```
dp[0][1] = 0
```

So:

```
Take = 1 + 0
      = 1
```

Therefore:

```
dp[3][2] = max(1, 1)
          = 1
```

Calculate dp[3][1]

Don't take:

```
dp[3][1] = 1
```

Take:

```
1 + dp[0][0]
= 1
```

Therefore:

```
dp[3][1] = 1
```

6. DP table after "0001"

Nothing actually changes:

		ONES →		
		0	1	2
ZERO	0	0	0	0
	1	0	1	1
	2	0	1	1
	3	0	1	1

Why couldn't we get 2?

Because:

```
"10" → 1 zero + 1 one
"0001" → 3 zeros + 1 one
```

Together:

```
zeros = 1 + 3 = 4
```

But we only have:

```
3 zeros
```

So both cannot be selected.

7. Process "1"

Now:

```
s = "1"
zeros = 0
ones = 1
```

Transition:

```
C++
dp[z][o] = max(
    dp[z][o],
    1 + dp[z][o - 1]
);
```

Because the string uses:

```
0 zeros
1 one
```

We loop:

```
z = 3 → 0
o = 2 → 1
```

Calculate $dp[3][2]$

Don't take:

```
dp[3][2] = 1
```

Take:

```
1 + dp[3][1]
```

We have:

```
dp[3][1] = 1
```

Therefore:

```
Take = 1 + 1
      = 2
```

So:

```
dp[3][2] = max(1, 2)
          = 2
```

This means we can take:

```
"10"
"1"
```

Resources:

```
zeros = 1 + 0 = 1
ones  = 1 + 1 = 2
```

Within:

```
zeros ≤ 3
ones  ≤ 2
```

Calculate $dp[3][1]$

Don't take:

```
1
```

Take:

```
1 + dp[3][0]
```

$dp[3][0] = 0$, so:

```
1 + 0 = 1
```

Therefore:

```
dp[3][1] = 1
```

Calculate $dp[2][2]$

```
Don't take = 1
```

```
Take:
1 + dp[2][1]
```

```
dp[2][1] = 1
```

```
Take = 2
```

Therefore:

```
dp[2][2] = 2
```

Calculate dp[1][2]

```
Don't take = 1
Take:
1 + dp[1][1]
dp[1][1] = 1
Take = 2
```

Therefore:

```
dp[1][2] = 2
```

8. DP table after "1"

		ONES →		
		0	1	2
ZERO	0	0	1	1
	1	0	1	2
	2	0	1	2
	3	0	1	2

Notice the important cell:

```
dp[3][2] = 2
```

9. Process "0"

Last string:

```
s = "0"
zeros = 1
ones = 0
```

Transition:

```
C++
dp[z][o] = max(
    dp[z][o],
    1 + dp[z - 1][o]
);
```

Because "0" consumes:

```
1 zero
0 ones
```

10. Calculate dp[3][2]

Don't take:

```
dp[3][2] = 2
```

Take:

```
1 + dp[2][2]
```

We have:

```
dp[2][2] = 2
```

Therefore:

```
Take = 1 + 2
      = 3
```

So:

```
dp[3][2] = max(2, 3)
           = 3
```

🎯 We found 3 strings!

Which ones?

```
"10"
"1"
"0"
```

Resource usage:

```
"10" → 1 zero, 1 one
"1"  → 0 zeros, 1 one
"0"  → 1 zero, 0 ones
-----
TOTAL → 2 zeros, 2 ones
```

We have:

```
3 zeros available
2 ones available
```

So it's valid.

11. Calculate the remaining important cells

dp[3][1]

```
Don't take = 1

Take:
1 + dp[2][1]
= 1 + 1
= 2

dp[3][1] = 2
```

dp[2][2]

```
Don't take = 2

Take:
1 + dp[1][2]
= 1 + 2
= 3

dp[2][2] = 3
```

dp[2][1]

```
Don't take = 1

Take:
1 + dp[1][1]
= 1 + 1
= 2

dp[2][1] = 2
```

dp[1][2]

```
Don't take = 2

Take:
1 + dp[0][2]
= 1 + 1
= 2

dp[1][2] = 2
```

12. FINAL DP TABLE

After processing all four strings:

		ONES →		
		0	1	2
ZERO	0	0	1	1
	1	1	2	2

2	1	2	3
3	1	2	3

The answer is:

```
C++
return dp[m][n];
```

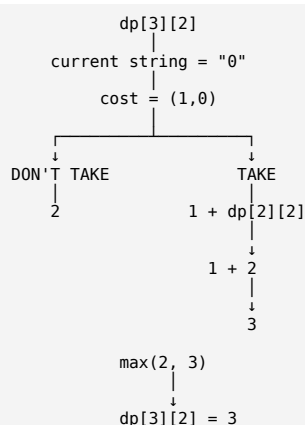
Therefore:

```
dp[3][2] = 3
```

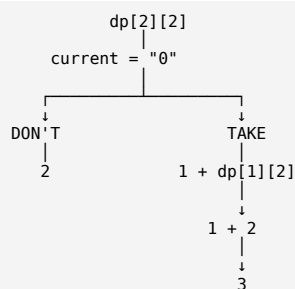
Answer = 3

13. The most important transition in this dry run

The final 3 came from this:



And $dp[2][2] = 2$ came from:



So the chain is essentially:

```

dp[3][2]
↓
take "0"
↓
1 + dp[2][2]
↓
1 + 2
↓
3
```

And the **three selected strings** are:

"10"	→ 1 zero, 1 one
"1"	→ 0 zeros, 1 one
"0"	→ 1 zero, 0 ones

↓
2 zeros, 2 ones
↓
3 strings

So although we have capacity for **3 zeros and 2 ones**, we don't need to use all the zeros. We only need to satisfy the constraint **at most** m zeros and **at most** n ones.

Final answer = 3.

ChatGPT can make mistakes. Check important info.